

Day 2: Getting the Data

Today's Agenda

- All about getting some data
- We'll be discussing two primary ways to getting at data:
 - via API
 - via Web Scraping

Survey Feedback

- You seemed to enjoy yesterday

Survey Feedback

- You seemed to enjoy yesterday
- That's great!

Survey Feedback

- You seemed to enjoy yesterday
- That's great!
- I only got one negative feedback...

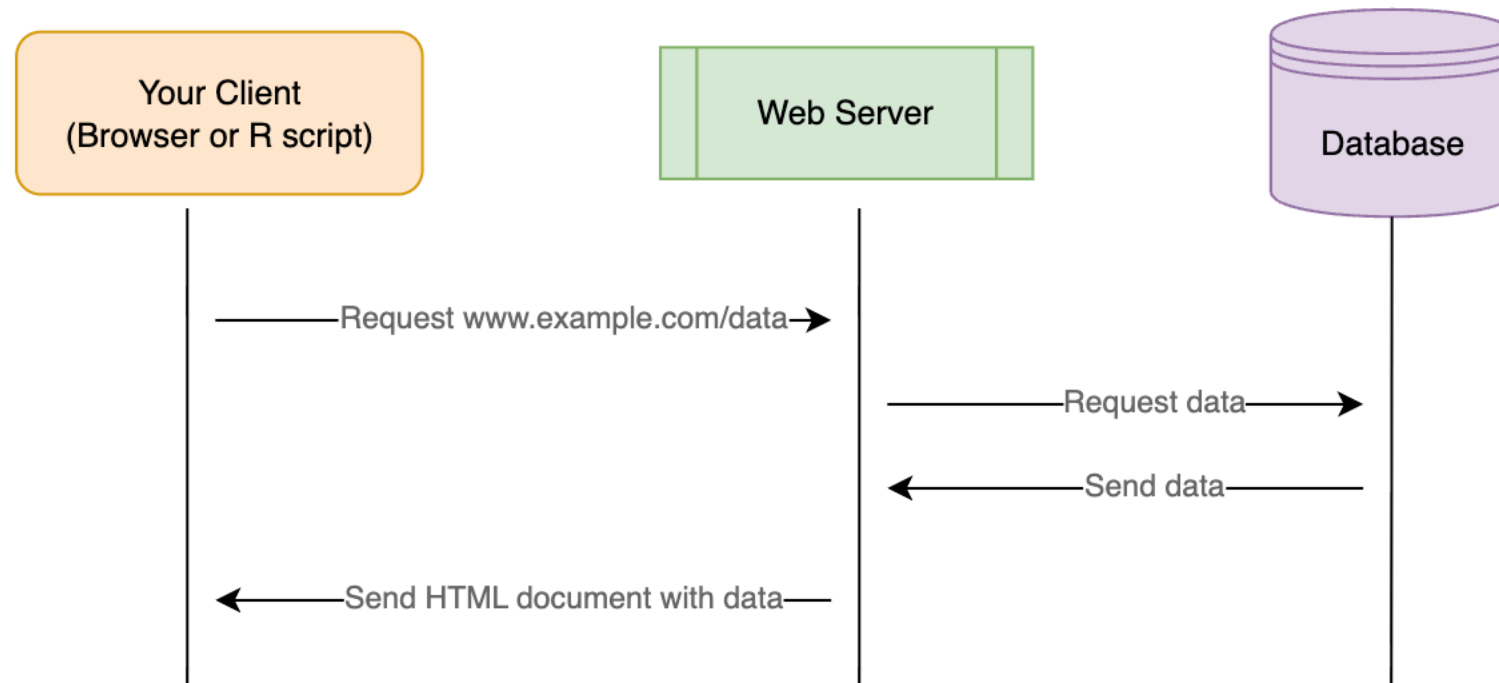
Survey Feedback

- You seemed to enjoy yesterday
- That's great!
- I only got one negative feedback...
- **I talk too fast**

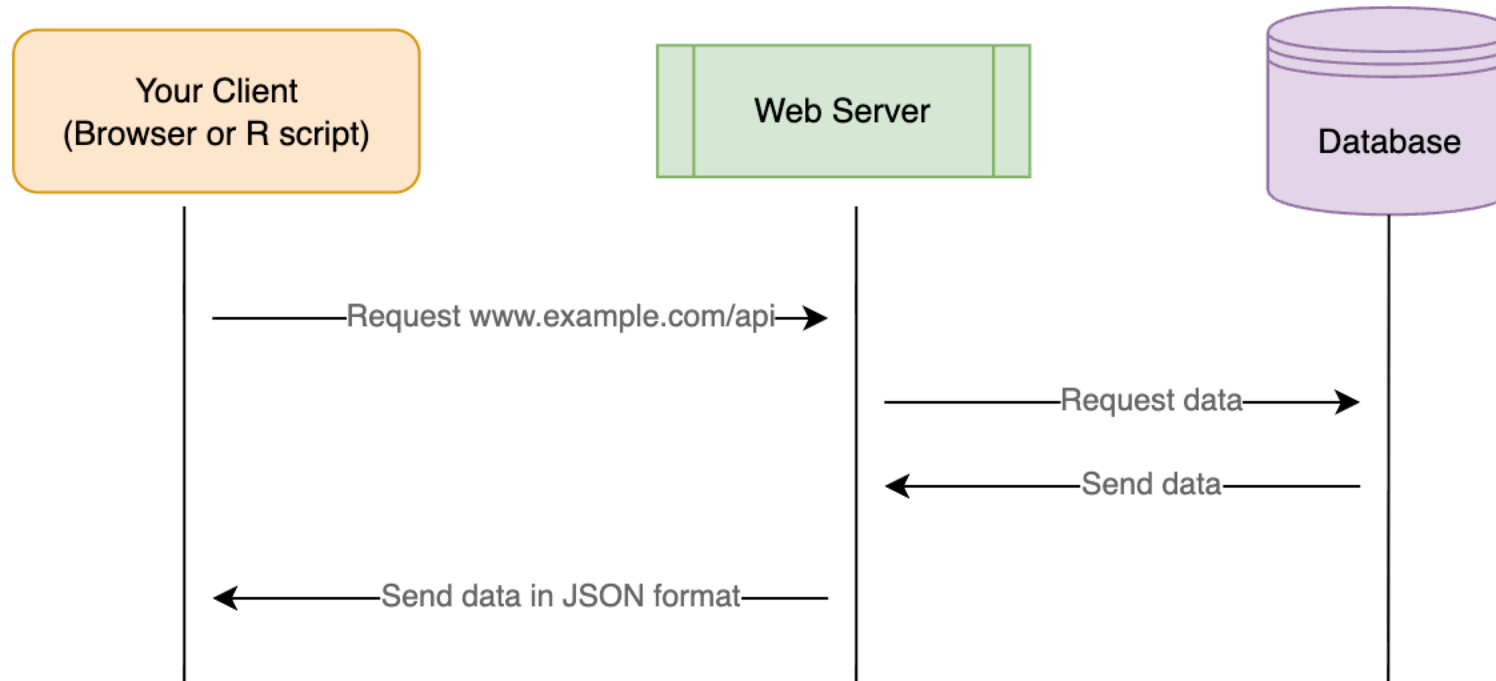
Survey Feedback

- You seemed to enjoy yesterday
 - That's great!
 - I only got one negative feedback...
 - **I talk too fast**
- (Also, some requested to have the slides available already for the lecture → please see the LISAM course room ;)

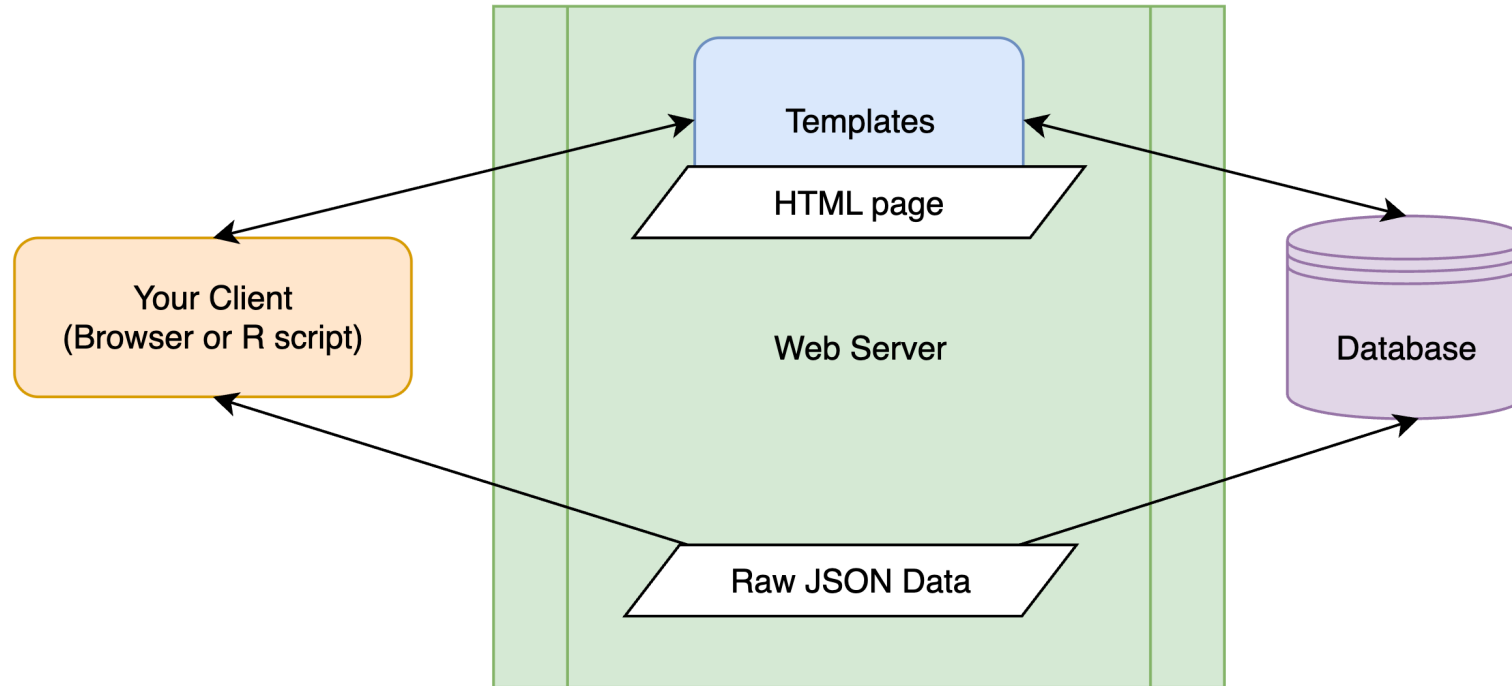
Reminder: How Visiting a Website Works



How Using an API Works



Visiting a Website vs an API



API vs Web Scraping

- Both getting data via API and via Scraping works exactly the same
 - The difference is in the endpoint (e.g., `/data` vs. `/api`) and the format the data comes back in (HTML vs. JSON)
- But how do we download websites, exactly...?

HTTP (HyperText Transfer Protocol)

Understanding the basis of "the internet"

HyperText Transfer Protocol

- HTTP is the protocol that your browser (or R) and a web server use to exchange data
 - Client and server establish a TCP connection
 - Then, the client sends a request
 - The server processes the request
 - The server sends a response to the client
- HTTP is **insecure**. That's why we usually use HTTPS (HTTP secure)
- HTTPS requires **certificates**
- It adds another step to the request: After establishment of the connection, the server must identify itself with a certificate. The client checks this certificate for authenticity.
- If the checks succeed, all data going through is **encrypted**

HyperText Transfer Protocol

- Important Concepts:
 - Request and Response
 - Status codes
 - HTTP “verbs”
 - Headers
 - “Body”



HTTP Request

- A client always sends a request to a server
- The request indicates to the server what the client wants to see

Example GET request:

```
GET / HTTP/1.1
```

```
Host: www.example.com
```

```
User-Agent: Mozilla/5.0
```

```
Accept-Language: en-GB,en;q=0.5
```

```
Accept-Encoding: gzip, deflate, br
```

```
Connection: keep-alive
```

HTTP Request

- A client always sends a request to a server
- The request indicates to the server what the client wants to see

Example POST request:

```
POST /test/demo_form.php
HTTP/1.1
Host: w3schools.com
```

```
name1=value1&name2=value2
```

HTTP Verbs

- An HTTP “verb” describes an “action” you wish to perform
- There are nine:
 - GET, HEAD, POST, PUT, DELETE, CONNECT, OPTIONS, TRACE, PATCH

HTTP Verbs

- An HTTP “verb” describes an “action” you wish to perform
- There are nine:
 - GET, HEAD, POST, PUT, DELETE, CONNECT, OPTIONS, TRACE, PATCH
- GET: Simply requests a website using a URL. Nothing more.
- POST: Request a website, but also provide some data using what is known as a “body”

HTTP Verbs

- An HTTP “verb” describes an “action” you wish to perform
- There are nine:
 - **GET**, HEAD, **POST**, PUT, DELETE, CONNECT, OPTIONS, TRACE, PATCH

Example GET request:

GET / HTTP/1.1

Host: www.example.com

User-Agent: Mozilla/5.0

Accept-Language: en-GB,en;q=0.5

Accept-Encoding: gzip, deflate, br

Connection: keep-alive

HTTP Verbs

- An HTTP “verb” describes an “action” you wish to perform
- There are nine:
 - **GET**, HEAD, **POST**, PUT, DELETE, CONNECT, OPTIONS, TRACE, PATCH

Example POST request:

```
POST /test/demo_form.php HTTP/1.1  
Host: w3schools.com
```

```
name1=value1&name2=value2
```

HTTP Response

- Every request is followed by a response by the server
- The server tells you whether the request was successful, or not, add some additional information that may be useful, and send the resource you requested.

Example GET response:

```
HTTP/1.1 200 OK
Date: Mon, 23 May 2005 22:38:34 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 155
Last-Modified: Wed, 08 Jan 2003 23:11:55 GMT
Server: Apache/1.3.3.7 (Unix) (Red-Hat/Linux)
ETag: "3f80f-1b6-3e1cb03b"
Accept-Ranges: bytes
Connection: close
```

```
<html>
  <head>
    <title>An Example Page</title>
  </head>
  <body>
    <p>Hello World, this is a very simple HTML
document.</p>
  </body>
</html>
```

HTTP Response

- Every request is followed by a response by the server
- The server tells you whether the request was successful, or not, add some additional information that may be useful, and send the resource you requested.

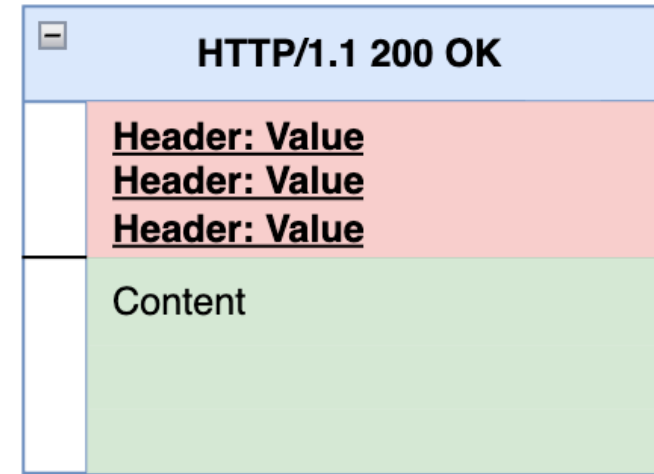
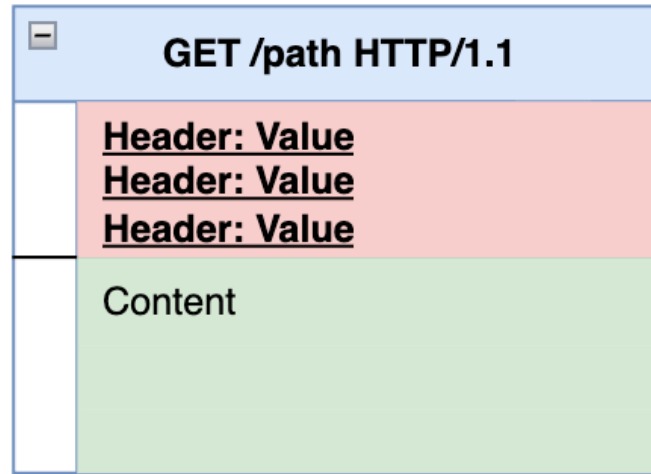
Protocol
Status Code
Headers
Body

Example GET response:

```
HTTP/1.1 200 OK
Date: Mon, 23 May 2005 22:38:34 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 155
Last-Modified: Wed, 08 Jan 2003 23:11:55 GMT
Server: Apache/1.3.3.7 (Unix) (Red-Hat/Linux)
ETag: "3f80f-1b6-3e1cb03b"
Accept-Ranges: bytes
Connection: close

<html>
  <head>
    <title>An Example Page</title>
  </head>
  <body>
    <p>Hello World, this is a very simple HTML
document.</p>
  </body>
</html>
```

HTTP Request & Response

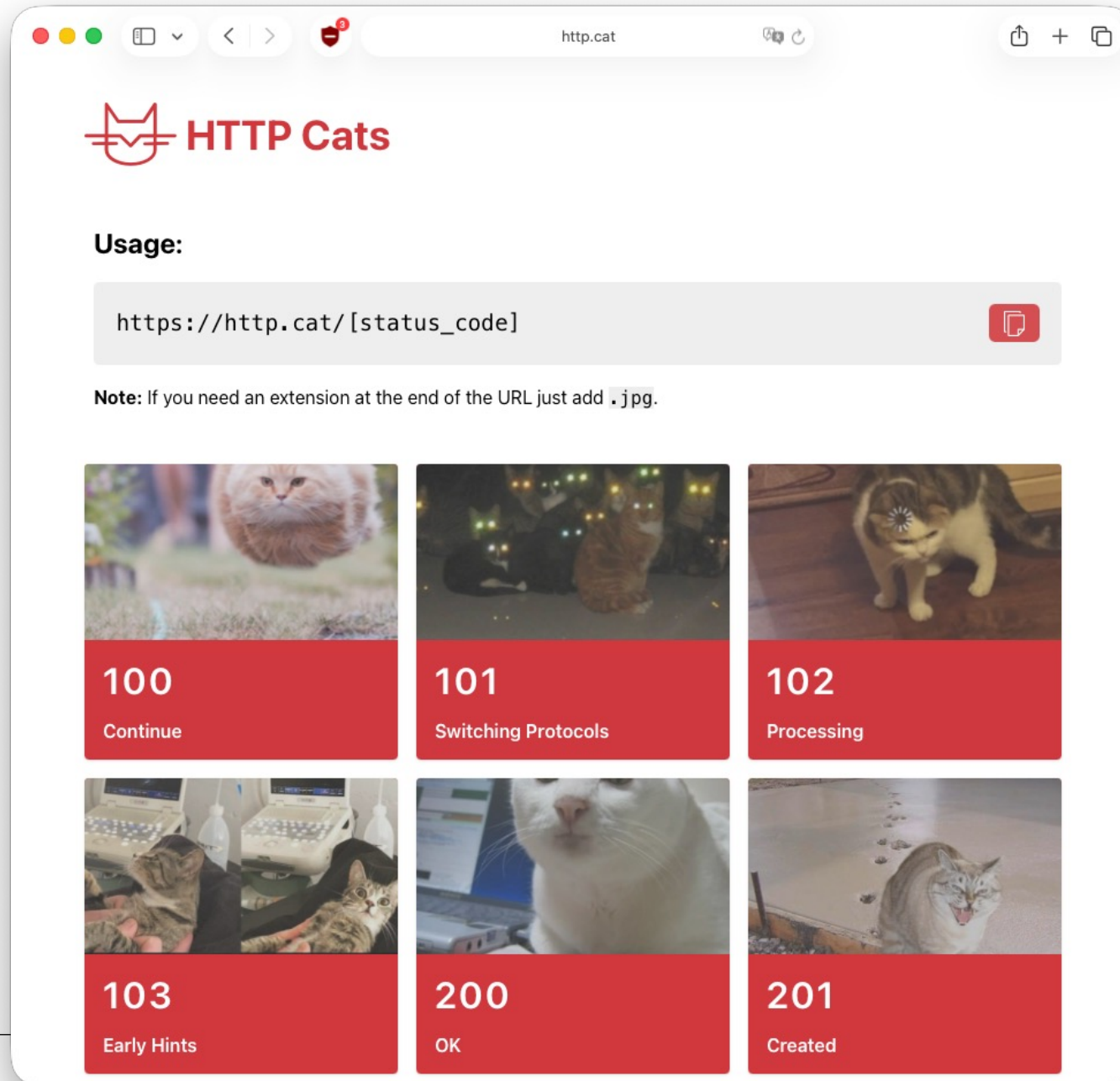


HTTP Status Codes

- The status code tells you whether your request was successful or if something happened
 - 1xx => Informative
 - 200 => All good
 - 3xx => Redirection
 - 4xx => You made an error
 - 5xx => The server made an error

```
> response$status_code  
[1] 200
```

HTTP Status Codes



HTTP Body

- The actual response follows after the headers
- That body is what contains, e.g., the HTML or whatever data the server sent back to you
 - If you query an API, the body will contain some JSON-encoded data
 - If you query a regular website, the body contains the HTML

Example GET response:

HTTP/1.1 200 OK

Date: Mon, 23 May 2005 22:38:34 GMT

Content-Type: text/html; charset=UTF-8

Content-Length: 155

Last-Modified: Wed, 08 Jan 2003 23:11:55 GMT

Server: Apache/1.3.3.7 (Unix) (Red-Hat/Linux)

ETag: "3f80f-1b6-3e1cb03b"

Accept-Ranges: bytes

Connection: close

```
<html>
  <head>
    <title>An Example Page</title>
  </head>
  <body>
    <p>Hello World, this is a very simple HTML
document.</p>
  </body>
</html>
```

Downloading Websites

Downloading Websites

- Now that we understand *how* your computer can download websites, it is time to do so!
- In R, this is done using, e.g., `httr`
- Alternative: `rvest`

```
library(httr)
url ← 'https://en.wikipedia.org/wiki/Web_scraping'
response ← GET(url)
```

Downloading Websites

- As you can see, we request a website using `GET()` which describes exactly what this will do.
- The web server then sends back a response

```
library(httr)  
url ← 'https://en.wikipedia.org/wiki/Web_scraping'  
response ← GET(url)
```

Downloading Websites

- As you can see, we request a website using `GET()` which describes exactly what this will do.
- The web server then sends back a response
- The response contains every piece of information imaginable

```
response List of 10
$ url : chr "https://en.wikipedia.org/wiki/Web_scraping"
$ status_code: int 200
$ headers :List of 29
.. $ date : chr "Sun, 12 Apr 2026 20:06:50 GMT"
.. $ server : chr "ATS/9.2.11"
.. $ x-content-type-options : chr "nosniff"
.. $ content-language : chr "en"
.. $ accept-ch : chr ""
.. $ last-modified : chr "Thu, 09 Apr 2026 19:39:06 GMT"
.. $ content-type : chr "text/html; charset=UTF-8"
.. $ content-encoding : chr "gzip"
.. $ age : chr "63775"
.. $ accept-ranges : chr "bytes"
.. $ x-cache : chr "cp1110 miss, cp1110 hit/42"
.. $ x-cache-status : chr "hit-front"
.. $ server-timing : chr "cache;desc=\"hit-front\", host;desc=\"cp1110\""
.. $ strict-transport-security : chr "max-age=106384710; includeSubDomains; preload"
.. $ report-to : chr "{ \"group\": \"wm_nel\", \"max_age\": 604800, \"endpoints\": [{ \"url\"...
.. $ nel : chr "{ \"report_to\": \"wm_nel\", \"max_age\": 604800, \"failure_fraction\":...
.. $ set-cookie : chr "WMF-Last-Access=13-Apr-2026;Path=/;HttpOnly;secure;Expires=Fri, 15 May ...
.. $ set-cookie : chr "WMF-Last-Access-Global=13-Apr-2026;Path=/;Domain=.wikipedia.org;HttpOnly...
.. $ set-cookie : chr "WMF-DP=8a4;Path=/;HttpOnly;secure;Expires=Tue, 14 Apr 2026 00:00:00 GMT"
.. $ x-client-ip : chr "2003:c6:6720:5500:f979:1d80:83fd:f748"
.. $ cache-control : chr "private, s-maxage=0, max-age=0, must-revalidate, no-transform"
.. $ content-security-policy : chr "default-src 'unsafe-eval' 'unsafe-inline' 'self' data: blob: *.wikimedi...
.. $ content-security-policy-report-only : chr "script-src 'unsafe-eval' blob: 'self' meta.wikimedia.org *.wikimedia.or...
.. $ vary : chr "Accept-Encoding,X-Subdomain,Cookie,Authorization,User-Agent"
.. $ set-cookie : chr "GeoIP=DE:NW:Euskirchen:50.66:6.79:v4; Path=/; secure; Domain=.wikipedia...
.. $ set-cookie : chr "NetworkProbeLimit=0.001;Path=/;Secure;SameSite=None;Max-Age=3600"
.. $ set-cookie : chr "WMF-Uniq=9yhnKjFRn1oldAlfJXJ-LgNBAAAAFvdqFOX3etgiV_YakzRkyqI_iiYKnd7JPO...
.. $ content-length : chr "37344"
.. $ x-request-id : chr "42246af7-a5c4-4d7f-9a10-7509e9d99ce6"
.. - attr(*, "class")= chr [1:2] "insensitive" "list"
$ all_headers:List of 1
.. $ :List of 3
.. .. $ status : int 200
.. .. $ version: chr "HTTP/2"
.. .. $ headers:List of 29
.. .. .. $ date : chr "Sun, 12 Apr 2026 20:06:50 GMT"
```

Downloading Websites

- The response includes everything that pertains to it:
 - Headers (e.g., cookies, response data, type, length, encoding, etc.)
 - A status code (e.g., 200)
 - And a body

```
response List of 10
$ url : chr "https://en.wikipedia.org/wiki/Web_scraping"
$ status_code: int 200
$ headers :List of 29
.. $ date : chr "Sun, 12 Apr 2026 20:06:50 GMT"
.. $ server : chr "ATS/9.2.11"
.. $ x-content-type-options : chr "nosniff"
.. $ content-language : chr "en"
.. $ accept-ch : chr ""
.. $ last-modified : chr "Thu, 09 Apr 2026 19:39:06 GMT"
.. $ content-type : chr "text/html; charset=UTF-8"
.. $ content-encoding : chr "gzip"
.. $ age : chr "63775"
.. $ accept-ranges : chr "bytes"
.. $ x-cache : chr "cp1110 miss, cp1110 hit/42"
.. $ x-cache-status : chr "hit-front"
.. $ server-timing : chr "cache;desc=\"hit-front\", host;desc=\"cp1110\""
.. $ strict-transport-security : chr "max-age=106384710; includeSubDomains; preload"
.. $ report-to : chr "{ \"group\": \"wm_nel\", \"max_age\": 604800, \"endpoints\": [{ \"url\"...
.. $ nel : chr "{ \"report_to\": \"wm_nel\", \"max_age\": 604800, \"failure_fraction\":...
.. $ set-cookie : chr "WMF-Last-Access=13-Apr-2026;Path=/;HttpOnly;secure;Expires=Fri, 15 May ...
.. $ set-cookie : chr "WMF-Last-Access-Global=13-Apr-2026;Path=/;Domain=.wikipedia.org;HttpOnl...
.. $ set-cookie : chr "WMF-DP=8a4;Path=/;HttpOnly;secure;Expires=Tue, 14 Apr 2026 00:00:00 GMT"
.. $ x-client-ip : chr "2003:c6:6720:5500:f979:1d80:83fd:f748"
.. $ cache-control : chr "private, s-maxage=0, max-age=0, must-revalidate, no-transform"
.. $ content-security-policy : chr "default-src 'unsafe-eval' 'unsafe-inline' 'self' data: blob: *.wikimedi...
.. $ content-security-policy-report-only : chr "script-src 'unsafe-eval' blob: 'self' meta.wikimedia.org *.wikimedia.or...
.. $ vary : chr "Accept-Encoding,X-Subdomain,Cookie,Authorization,User-Agent"
.. $ set-cookie : chr "GeoIP=DE:NW:Euskirchen:50.66:6.79:v4; Path=/; secure; Domain=.wikipedia...
.. $ set-cookie : chr "NetworkProbeLimit=0.001;Path=/;Secure;SameSite=None;Max-Age=3600"
.. $ set-cookie : chr "WMF-Uniq=9yhnKjFRn1oldAlfJXJ-LgNBAAAAAFvdqFOX3etgiV_YakzRkyqI_iiYKnd7JPO...
.. $ content-length : chr "37344"
.. $ x-request-id : chr "42246af7-a5c4-4d7f-9a10-7509e9d99ce6"
.. - attr(*, "class")= chr [1:2] "insensitive" "list"
$ all_headers:List of 1
.. $ :List of 3
.. .. $ status : int 200
.. .. $ version: chr "HTTP/2"
.. .. $ headers:List of 29
.. .. .. $ date : chr "Sun, 12 Apr 2026 20:06:50 GMT"
```

Downloading Websites

- The `content` property contains the actual results
- Can someone tell me what we see here?

```
> response$content
[1] 3c 21 44 4f 43 54 59 50 45 20 68 74 6d 6c 3e 0a 3c 68 74 6d 6c 20 63 6c 61
[55] 65 2d 6c 61 6e 67 75 61 67 65 2d 69 6e 2d 68 65 61 64 65 72 2d 65 6e 61 62
[109] 6e 2d 6d 61 69 6e 2d 6d 65 6e 75 2d 64 69 73 61 62 6c 65 64 20 76 65 63 74
[163] 61 67 65 2d 68 65 61 64 65 72 2d 64 69 73 61 62 6c 65 64 20 76 65 63 74 6f
[217] 69 73 61 62 6c 65 64 20 76 65 63 74 6f 72 2d 66 65 61 74 75 72 65 2d 74 6f
[271] 66 65 61 74 75 72 65 2d 6d 61 69 6e 2d 6d 65 6e 75 2d 70 69 6e 6e 65 64 2d
[325] 65 64 2d 77 69 64 74 68 2d 63 6c 69 65 6e 74 70 72 65 66 2d 31 20 76 65 63
[379] 74 65 6e 74 2d 65 6e 61 62 6c 65 64 20 76 65 63 74 6f 72 2d 66 65 61 74 75
[433] 66 2d 31 20 76 65 63 74 6f 72 2d 66 65 61 74 75 72 65 2d 61 70 70 65 61 72
[487] 2d 74 68 65 6d 65 2d 63 6c 69 65 6e 74 70 72 65 66 2d 64 61 79 20 76 65 63
[541] 74 6f 72 2d 74 6f 63 2d 61 76 61 69 6c 61 62 6c 65 20 73 6b 69 6e 2d 74 68
[595] 22 20 6c 61 6e 67 3d 22 65 6e 22 20 64 69 72 3d 22 6c 74 72 22 3e 0a 3c 68
[649] 74 69 74 6c 65 3e 57 65 62 20 73 63 72 61 70 69 6e 67 20 2d 20 57 69 6b 69
[703] 6f 6e 28 29 7b 76 61 72 20 63 6c 61 73 73 4e 61 6d 65 3d 22 63 6c 69 65 6e
```

Downloading Websites

- The content is typically returned as raw data, and displayed as **hexadecimal**
- To create text from it, we need the `content` function.
- What you can see here is HTML text.

```
> content(response, as = 'text')
[1] "<!DOCTYPE html>\n<html class=\"client-nojs vector-feature-language-in-hea
r-disabled vector-feature-page-tools-pinned-disabled vector-feature-toc-pinne
tor-feature-limited-width-content-enabled vector-feature-custom-font-size-clie
r-enabled vector-toc-available skin-theme-clientpref-thumb-standard\" lang=\"e
>(function(){var className=\"client-js vector-feature-language-in-header-enabl
d vector-feature-page-tools-pinned-disabled vector-feature-toc-pinned-clientpf
re-limited-width-content-enabled vector-feature-custom-font-size-clientpref-1
vector-toc-available skin-theme-clientpref-thumb-standard\";var cookie=document
h(function(pref){className=className.replace(new RegExp('(^\| )'+pref.replace(,
ment.className=className;})();RLCONF={\"wgBreakFrames\":false,\"wgSeparatorTra
MonthNames\":[\"\", \"January\", \"February\", \"March\", \"April\", \"May\", \"June
82-7f37-45b5-9205-5dc89df2e66a\", \"wgCanonic ... <truncated>
```

Excursus: Encoding

How computers interpret numbers

Encoding

- Computers work in binary
- That means they only store 1 or 0
- Ones and zeros are combined into groups of eight
 - one 1/0 is a bit, eight are a byte
- → These are just regular numbers

00000110 → 6

00000001 → 1

10000000 → 127

11111111 → 255

Encoding

- All *text* that you see on screen is just *numbers* for your computer!
- → In order to turn numbers into text, your computer needs a **character map**

```
> res
[1] 3c 21 44 4f 43 54 59 50 45 20 68 74 6d 6c 3e 0a 3c 68 74 6d 6c 20 63 6c 61
[55] 65 2d 69 6e 2d 68 65 61 64 65 72 2d 65 6e 61 62
[109] 6e 2d 6d 61 69 6e 2d 6d 65 6e 75 2d 64 69 73 61 62 6c 65 64 20 76 65 63 74
[163] 61 67 65 2d 68 65 61 64 65 72 2d 64 69 73 61 62 6c 65 64 20 76 65 63 74 6f
[217] 69 73 61 62 6c 65 64 20 76 65 63 74 6f 72 2d 66 65 61 74 75 72 65 2d 74 6f
[271] 66 65 61 74 75 72 65 2d 6d 61 69 6e 2d 6d 65 6e 75 2d 70 69 6e 6e 65 64 2d
[325] 65 64 2d 77 69 64 74 68 2d 63 6c 69 65 6e 74 70 72 65 66 2d 31 20 76 65 63
[379] 74 65 6e 74 2d 65 6e 61 62 6c 65 64 20 76 65 63 74 6f 72 2d 66 65 61 74 75
[433] 66 2d 31 20 76 65 63 74 6f 72 2d 66 65 61 74 75 72 65 2d 61 70 70 65 61 72
[487] 2d 74 68 65 6d 65 2d 63 6c 69 65 6e 74 70 72 65 66 2d 64 61 79 20 76 65 63
[541] 74 6f 72 2d 74 6f 63 2d 61 76 61 69 6c 61 62 6c 65 20 73 6b 69 6e 2d 74 68
[595] 22 20 6c 61 6e 67 3d 22 65 6e 22 20 64 69 72 3d 22 6c 74 72 22 3e 0a 3c 68
[649] 74 69 74 6c 65 3e 57 65 62 20 73 63 72 61 70 69 6e 67 20 2d 20 57 69 6b 69
[703] 6f 6e 28 29 7b 76 61 72 20 63 6c 61 73 73 4e 61 6d 65 3d 22 63 6c 69 65 6e
```

Encoding

- All *text* that you see on screen is just *numbers* for your computer!
- → In order to turn numbers into text, your computer needs a **character map**
- Common ones are ASCII or UTF-8

```
> res
[1] 3c 21 44 4f 43 54 59 50 45 20 68 74 6d 6c 3e 0a 3c 68 74 6d 6c 20 63 6c 61
[55] 65 2d 69 6e 2d 68 65 61 64 65 72 2d 65 6e 61 62
[109] 6e 2d 6d 61 69 6e 2d 6d 65 6e 75 2d 64 69 73 61 62 6c 65 64 20 76 65 63 74
[163] 61 67 65 2d 68 65 61 64 65 72 2d 64 69 73 61 62 6c 65 64 20 76 65 63 74 6f
[217] 69 73 61 62 6c 65 64 20 76 65 63 74 6f 72 2d 66 65 61 74 75 72 65 2d 74 6f
[271] 66 65 61 74 75 72 65 2d 6d 61 69 6e 2d 6d 65 6e 75 2d 70 69 6e 6e 65 64 2d
[325] 65 64 2d 77 69 64 74 68 2d 63 6c 69 65 6e 74 70 72 65 66 2d 31 20 76 65 63
[379] 74 65 6e 74 2d 65 6e 61 62 6c 65 64 20 76 65 63 74 6f 72 2d 66 65 61 74 75
[433] 66 2d 31 20 76 65 63 74 6f 72 2d 66 65 61 74 75 72 65 2d 61 70 70 65 61 72
[487] 2d 74 68 65 6d 65 2d 63 6c 69 65 6e 74 70 72 65 66 2d 64 61 79 20 76 65 63
[541] 74 6f 72 2d 74 6f 63 2d 61 76 61 69 6c 61 62 6c 65 20 73 6b 69 6e 2d 74 68
[595] 22 20 6c 61 6e 67 3d 22 65 6e 22 20 64 69 72 3d 22 6c 74 72 22 3e 0a 3c 68
[649] 74 69 74 6c 65 3e 57 65 62 20 73 63 72 61 70 69 6e 67 20 2d 20 57 69 6b 69
[703] 6f 6e 28 29 7b 76 61 72 20 63 6c 61 73 73 4e 61 6d 65 3d 22 63 6c 69 65 6e
```

ASCII

(American Standard Code for Information Interchange)

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	00	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	01	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	02	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	03	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	04	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	05	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	06	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	07	[BELL]	39	27	'	71	47	G	103	67	g
8	08	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	09	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	0A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	0B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	0C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	0D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	0E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	0F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Encoding

- Let's try it with the first numbers of the response content:
- 3c 21 44 4f 43 54 59 50 45

Hexadecimal		
3c		
21		
44		
4f		
43		
54		
59		
50		
45		

Encoding

- Let's try it with the first numbers of the response content:
- 3c 21 44 4f 43 54 59 50 45
 - Side note: Hexadecimal is just a common way to represent numbers in computing.* It works like our decimal system (0-9), except that it has 16 digits (0-9 and a-f), hence *hexadecimal*.

Hexadecimal		
3c		
21		
44		
4f		
43		
54		
59		
50		
45		

Encoding

- Let's try it with the first numbers of the response content:
- 3c 21 44 4f 43 54 59 50 45
 - In decimal:
- 60 33 68 79 67 84 89 80 69

Hexadecimal	Decimal	
3c	60	
21	33	
44	68	
4f	79	
43	67	
54	84	
59	89	
50	80	
45	69	

ASCII

(American Standard Code for Information Interchange)

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	00	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	01	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	02	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	03	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	04	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	05	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	06	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	07	[BELL]	39	27	'	71	47	G	103	67	g
8	08	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	09	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	0A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	0B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	0C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	0D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	0E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	0F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Encoding

- Let's try it with the first numbers of the response content:
- 3c 21 44 4f 43 54 59 50 45
 - In decimal:
- 60 33 68 79 67 84 89 80 69
 - Converted via ASCII:
- < ! D O C T Y P E

Hexadecimal	Decimal	ASCII
3c	60	<
21	33	!
44	68	D
4f	79	O
43	67	C
54	84	T
59	89	Y
50	80	P
45	69	E

Encoding

- This is what the function `content(response, as = 'text')` does

```
> content(response, as = 'text')
[1] "<!DOCTYPE html>\n<html class=\"client-nojs vector-feature-language-in-hea
r-disabled vector-feature-page-tools-pinned-disabled vector-feature-toc-pinned
tor-feature-limited-width-content-enabled vector-feature-custom-font-size-clie
r-enabled vector-toc-available skin-theme-clientpref-thumb-standard\" lang=\"en
>(function(){var className=\"client-js vector-feature-language-in-header-enabl
d vector-feature-page-tools-pinned-disabled vector-feature-toc-pinned-clientpr
re-limited-width-content-enabled vector-feature-custom-font-size-clientpref-1
vector-toc-available skin-theme-clientpref-thumb-standard\";var cookie=document
h(function(pref){className=className.replace(new RegExp('(\\| )'+pref.replace(/
ment.className=className;})();RLCONF={\"wgBreakFrames\":false,\"wgSeparatorTri
MonthNames\":[\"\", \"January\", \"February\", \"March\", \"April\", \"May\", \"June
82-7f37-45b5-9205-5dc89df2e66a\", \"wgCanonic ... <truncated>
```

Encoding

- Important: Websites sometimes lie about the text encoding!
- They may use one encoding to write the HTML, but tell you another one (or none at all!)

Encoding

```

1  NUDNUDNUDNUDETxEACKj ШqCHI-oms/;ФChHaETDC3">wVqJ. _zA-YxHA@^9j@ENQDMЙZФq. &
  „Ove|ИyулZSOHe.X64STXC>@sJjJyU@...y+oПI6RBEHOY3METX»»=K/e>*DLEIXHЪJSРф»[Uhrp4NUD\ 'STXWMCTьk}nLE-~_A
  ACKYщрNULXIENUDNULИ;|=aSTX»»7+„EMB+rxDC3Spr- /И"%PMSOHe.X64STXC>@sJjJyU@...y+oПI6RBEHOY3METX»»=K/e>*
  DLEIXHЪJSРф»[Uhrp4NUD\ 'STXWMCTьk}nLE-~_AACKYщрNULXIENUDNULy, iFЙETX«?»\GZfhjXъMiSЙ[coatgen/760e6d67
  /jav_ctr_boulder_chaos_wide_b_b5c531c10f59b61eee214856a55849aa.dynamicsliceNUDy+EMxqCIYy3GHOИPDC1cyв""
  BSNUDNUDNUDNUDNUDETxEACKy«iK_6RBEHOY3METX»»=K/e>*DLEXъi••F:K+P"щACKщ+NUDNUDNUDHA@^9j@ENQDM
  ЙZФq. &
  „Ove|ИyулZSOHe.X64STXC>@sJjJyU@...y+oПI6RBEHOY3METX»»=K/e>*DLEIXHЪJSРф»[Uhrp4NUD\ 'STXWMCTьk}nLE-~_A
  ACKYщрNULXIENUDNULИ;|=aSTX»»7+„EMB+rxDC3Spr- /И"%PMSOHe.X64STXC>@sJjJyU@...y+oПI6RBEHOY3METX»»=K/e>*
  DLEIXHЪJSРф»[Uhrp4NUD\ 'STXWMCTьk}nLE-~_AACKYщрNULXIENUDNULy, iFЙETX«?»\GZfhjXъMiSЙcoatgen/760e6d67
  /jav_ctr_boulder_chaos_wide_b_b5c531c10f59b61eee214856a55849aa.slice.metaNUDy+EMxqCIYy3GHOИPDC1cyв""
  BSNUDNUDNUDNUDNUDETxEACKy«iK_6RBEHOY3METX»»=K/e>*DLENUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUD
  NUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUD
  „Ove|ИyулZSOHe.X64STXC>@sJjJyU@...y+oПI6RBEHOY3METX»»=K/e>*DLEQNUDETXXIIIтз е«ЦИЕТБЕЧqYNUD\ 'STXWMCTьk}nL
  E-~_AACKYщрNUDETXXNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUD
  ETX»»=K/e>*DLEQNUDETXXIIIтз е«ЦИЕТБЕЧqYNUD\ 'STXWMCTьk}nLE-~_AACKYщрNUDETXXNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUD
  fhjXъMiSЙaobjects/crafting/props/furniture/shelving/smallcabinets/textures/jav_prp_smallcabinets_spec.
  dd.3NUDy+EMxqCIYy3GHOИPDC1cyв"" BSNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUD
  NUDNUDy«iK_6RBEHOY3METX»»=K/e>*DLE9CANrKYKMHf>ИH Ж, RSNUDNUDNUDHA@^9j@ENQDMЙZФq. &
  „Ove|ИyулZSOHe.X64STXC>@sJjJyU@...y+oПI6RBEHOY3METX»»=K/e>*DLEЙNULSTX
  ФФPm± BSiMHeBNUL\ 'STXWMCTьk}nLE-~_AACKYщрNUDETXXNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUD
  64STXC>@sJjJyU@...y+oПI6RBEHOY3METX»»=K/e>*DLEЙNULSTX
  ФФPm± BSiMHeBNUL\ 'STXWMCTьk}nLE-~_AACKYщрNUDETXXNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUDNUD
  eXtensible Markup Language file length: 322 669 283 lines: 345 938 Ln: 1 Col: 1 Sel: 0|0 Macintosh (CR) ANSI INS

```

Encoding

[illegible]

Encoding

- Important: Websites sometimes lie about the text encoding!
- They may use one encoding to write the HTML, but tell you another one (or none at all!)

Recommended talk: Dylan Beattie – Plain Text



<https://youtu.be/4mRxIgu9R70>

Primer to HTML

HyperText Markup Language

- HTML is a variant of XML (eXtensible Markup Language)
- Consists of so-called “tags”
- Organized as a tree-structure



```
<!DOCTYPE html>
<html>
  <head>
    <title>This is a title</title>
  </head>
  <body>
    <div>
      <p>Hello world!</p>
    </div>
  </body>
</html>
```

HyperText Markup Language

- Root node: `<html>`
- `<head>`: Contains meta-information about the document (the title, a description, keywords)
- `<body>`: Contains the actual HTML contents → this is what you mostly see

```
<!DOCTYPE html>
<html>
  <head>
    <title>This is a title</title>
  </head>
  <body>
    <div>
      <p>Hello world!</p>
    </div>
  </body>
</html>
```

WIKIPEDIA25 years of the free encyclopedia

Search Wikipedia

Search

Donate

Create account

Log in

HTML

140 languages

Contents

hide

(Top)

> History

> Markup

Semantic HTML

> Delivery

> HTML4 variations

WHATWG HTML versus HTML5

WYSIWYG editors

See also

Notes

References

External links

Article

Talk

Read

View source

View history

Tools

From Wikipedia, the free encyclopedia

"*.htm*" and "*.html*" redirect here. For other uses, see *HTM*.

!

This article **contains instructions or advice**. Wikipedia is not a guidebook; please help **rewrite such content** to be encyclopedic or move it to **Wikiversity**, **Wikibooks**, or **Wikivoyage**. (November 2025)

HTML

HTML

Official logo of HTML5^[1]

Filename extension

.html

.htm

Internet media type

text/html

Type code

TEXT

Uniform Type Identifier (UTI)

public.html

Developed by

WHATWG

World Wide Web Consortium (W3C; formerly)

Initial release

1993; 33 years ago

Latest release

Living Standard

Type of format

Document file format

Container for

HTML elements

Contained by

Web browser

Extended from

SGML

Extended to

XHTML

Open format?

Yes

Website

html.spec.whatwg.org

Hypertext Markup Language (HTML) is the standard markup language^[a] for documents designed to be displayed in a web browser. It defines the content and structure of web content. It is often assisted by technologies such as Cascading Style Sheets (CSS) and scripting languages such as JavaScript.

Web browsers receive HTML documents from a web server or from local storage and render the documents into multimedia web pages. HTML describes the structure of a web page semantically and originally included cues for its appearance.

HTML elements are the building blocks of HTML pages. With HTML constructs, images and other objects such as interactive forms may be embedded into the rendered page. HTML provides a means to create structured documents by denoting structural semantics for text such as headings, paragraphs, lists, links, quotes, and other items. HTML elements are delineated by tags, written using angle brackets. Tags such as and <input> directly introduce content into the page. Other tags such as <p> and its corresponding closing tag </p> surround and provide information about document text and may include sub-element tags. Browsers do not display the HTML tags, but use them to interpret the content of the page.

Appearance

hide

Text

Small

Standard

Large

Width

Standard

Wide

Color (beta)

Automatic

Light

Dark

HTML - Wikipedia

This is the <title>

WIKIPEDIA25 years of the free encyclopedia

Search Wikipedia

Search

Donate

Create account

Log in

HTML

140 languages

Contents

hide

(Top)

> History

> Markup

Semantic HTML

> Delivery

> HTML4 variations

WHATWG HTML versus HTML5

WYSIWYG editors

See also

Notes

References

External links

Article

Talk

Read

View source

View history

Tools

From Wikipedia, the free encyclopedia

"*.htm*" and "*.html*" redirect here. For other uses, see *HTM*.

! This article contains instructions or advice. Wikipedia is not a guidebook; please help rewrite such content to be encyclopedic or move it to Wikiversity, Wikibooks, or Wikivoyage. (November 2025)

Hypertext Markup Language (HTML) is the standard markup language^[a] for documents designed to be displayed in a web browser. It defines the content and structure of web content. It is often assisted by technologies such as Cascading Style Sheets (CSS) and scripting languages such as JavaScript.

Web browsers receive HTML documents from a web server or from local storage and render the documents into multimedia web pages. HTML describes the structure of a web page semantically and originally included cues for its appearance.

HTML elements are the building blocks of HTML pages. With HTML constructs, images and other objects such as interactive forms may be embedded into the rendered page. HTML provides a means to create structured documents by denoting structural semantics for text such as headings, paragraphs, lists, links, quotes, and other items. HTML elements are delineated by tags, written using angle brackets. Tags such as and <input> directly introduce content into the page. Other tags such as <p> and its corresponding closing tag </p> surround and provide information about document text and may include element tags. Browsers do not display the HTML tags, but use them to interpret the content of the page.

HTML

HTML

Official logo of HTML5^[1]

Filename extension	.html .htm
Internet media type	text/html
Type code	TEXT
Uniform Type Identifier (UTI)	public.html
Developed by	WHATWG World Wide Web Consortium (W3C; formerly)
Initial release	1993; 33 years ago
Latest release	Living Standard
Type of format	Document file format
Container for	HTML elements
Contained by	Web browser
Extended from	SGML
Extended to	XHTML
Open format?	Yes
Website	html.spec.whatwg.org

Appearance

hide

Text

Small

Standard

Large

Width

Standard

Wide

Color (beta)

Automatic

Light

Dark

HTML - Wikipedia

HTML Elements

- There are **semantic** and **non-semantic** HTML tags
 - **Semantic** tags are semantic because they imply a specific meaning
 - **Non-semantic** tags are intended for general use and do not convey any implications about their contents

HTML Elements

- There are **semantic** and **non-semantic** HTML tags
 - **Semantic** tags are semantic because they imply a specific meaning
 - **Non-semantic** tags are intended for general use and do not convey any implications about their contents
- Examples of semantic tags:
 - `<h1>`: A heading level 1
 - `<p>`: A paragraph
 - `<ul>`: An unordered list
 - `<nav>`: A navigation bar
 - `<img>`: An image
 - `<a>`: An anchor (clickable link)
 - `<link>`: References a resource (not a clickable link!)

HTML Elements

- There are **semantic** and **non-semantic** HTML tags
 - **Semantic** tags are semantic because they imply a specific meaning
 - **Non-semantic** tags are intended for general use and do not convey any implications about their contents
- Examples of non-semantic tags:
 - `<div>`: Diverse, a generic container for various contents
 - `<span>`: An inline-element that usually “spans” some text

HTML Elements

- There are **semantic** and **non-semantic** HTML tags
 - **Semantic** tags are semantic because they imply a specific meaning
 - **Non-semantic** tags are intended for general use and do not convey any implications about their contents
- Examples of non-semantic tags:
 - `<div>`: Diverse, a generic container for various contents
 - `<span>`: An inline-element that usually “spans” some text
- → Most tags are semantic, but you will primarily find non-semantic tags on websites

HTML Elements

- Most tags require an opening and a closing tag:
 - `<p>This is a paragraph of text</p>`
- Some are “self-closing” (they do not have content):
 - `<img alt=“this is an image” />`
- HTML Elements can also have **attributes**:
 - `<a href=“https://www.google.com”>A link to Google</a>`
 - `<img src=“/my-image.jpg” alt=“Alternative text” />`
 - `<p class=“lead-paragraph”>Some text</p>`

HTML Elements

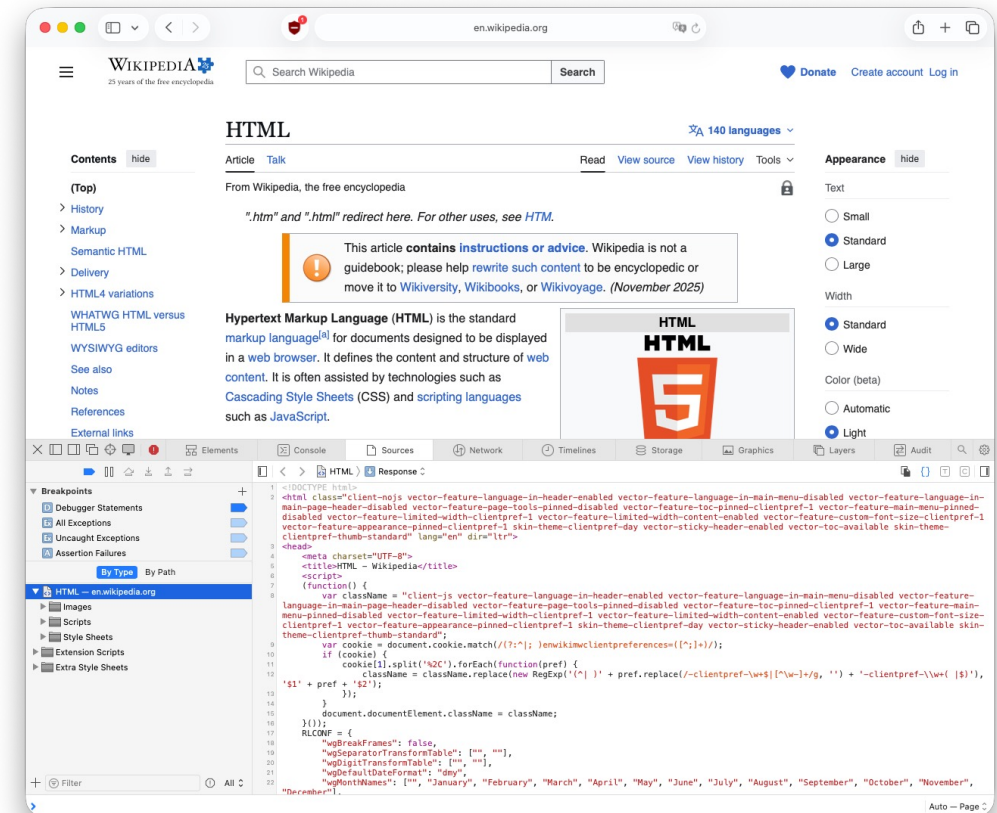
- Important: Website developers are extremely lazy people (I speak from experience)
- They make constant mistakes:
 - Forgetting to close a tag
 - Forgetting quotes around attributes
 - Nesting the tags incorrectly

Your browser will display this:

```
<!DOCTYPE html>
<html>
  <head>
    <title>This is a title</title>
  </head>
  <body>
    <div>
      <h1>My Website
      <p>Hello world!
      <p>This is a second paragraph
    </body>
  </html>
```

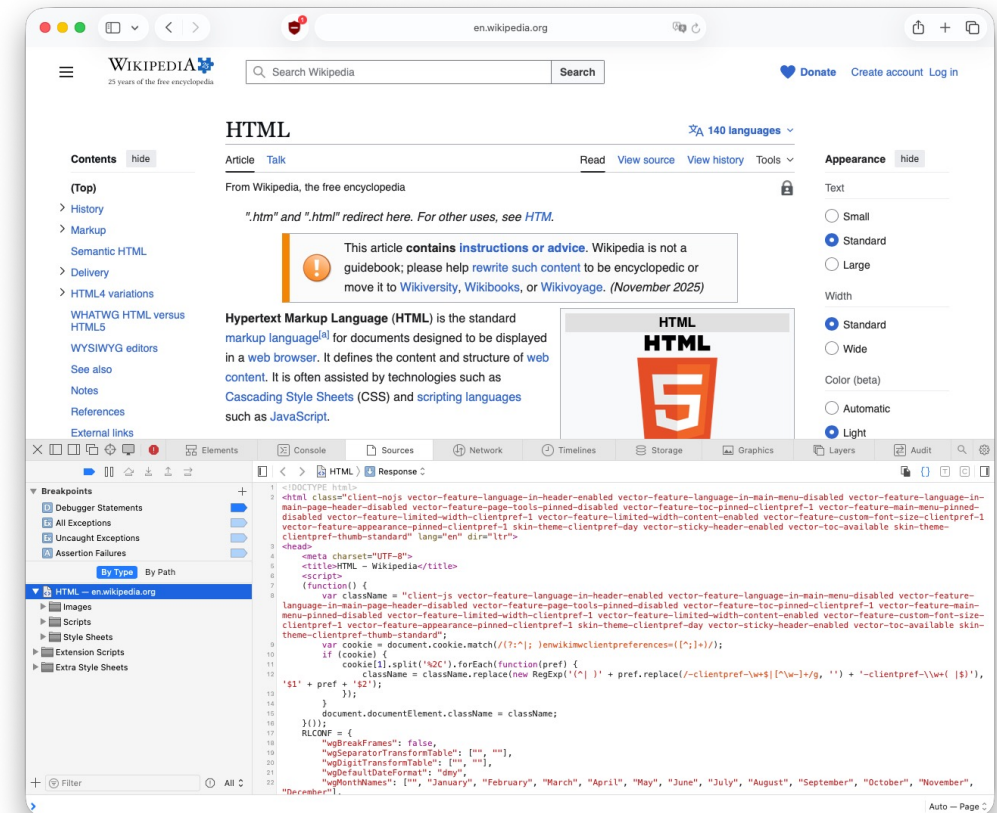
Viewing a Page's Source Code

- When you visit a website in your browser, the browser will parse and display the HTML to you
- To see the actual source code of a website, you'll need to “View Source Code” in the context menu of your browser



Viewing a Page's Source Code

- Most browsers also allow you to “inspect” an “element”
- This allows you to identify elements on the website and show where in the HTML tree they are



Viewing a Page's Source Code

The screenshot shows the Wikipedia article for HTML. The article text is on the left, and the developer tools are open on the right. The developer tools show the HTML source code in the top pane and the CSS styles in the bottom pane. The HTML source code shows the document structure, including the header, main content, and footer. The CSS styles show the default styles for the HTML element, including font-family, height, background-color, and color.

HTML

Article Talk

From Wikipedia, the free encyclopedia

"htm" and ".html" redirect here. For other uses, see *HTML*.

This article contains instructions or advice. Wikipedia is not a guidebook; please help rewrite such content to be encyclopedic or move it to Wikiversity, Wikibooks, or Wikivoyage. (November 2025)

Hypertext Markup Language (HTML) is the standard **markup language**^[a] for documents designed to be displayed in a **web browser**. It defines the content and structure of **web content**. It is often assisted by technologies such as **Cascading Style Sheets** (CSS) and **scripting languages** such as **JavaScript**.

Web browsers receive HTML documents from a **web server** or from local storage and **render** the documents into **multimedia** web pages. HTML describes the structure of a **web page** **semantically** and originally included cues for its appearance.

HTML **elements** are the building blocks of HTML pages. With HTML constructs, **images** and other objects such as **interactive forms** may be embedded into the rendered page. HTML provides a means to create **structured documents** by denoting structural **semantics** for text such as headings, paragraphs, lists, links, quotes, and other items. HTML elements are delineated by **tags**, written using **angle brackets**. Tags such as `<img>` and `<input>` directly introduce content into the page. Other tags such as `<p>` and its corresponding closing tag `</p>` surround and provide information about document text and may include sub-element tags. **Browsers** do not display the HTML tags, but use them to interpret the content of the page.

HTML can embed programs written in a **scripting language** such as **JavaScript**, which affects the behavior and content of web pages. The inclusion of CSS defines the look and layout of content. The **World Wide Web Consortium** (W3C), former maintainer of the HTML and current maintainer of the CSS standards, has encouraged the use of CSS over explicit presentational HTML since 1997.^[3] A form of HTML, known as **HTML5**, is used to display video and audio, primarily using the `<canvas>` element, together with JavaScript.

History

Development

In 1980, the physicist **Tim Berners-Lee**, a contractor at CERN, conceived and created **ENQUIRE**, a system for CERN researchers to

HTML

Official logo of HTML5^[1]

Filename extension `.html`
`.htm`

Internet media type `text/html`

Type code `TEXT`

Uniform Type Identifier (UTI) `public.html`

Developed by WHATWG
World Wide Web Consortium (W3C; formerly)

Initial release 1993; 33 years ago

Latest release Living Standard[?]

Type of format Document file format

Container for HTML elements

Extended from Web browser

Extended to SGML

Open format? Yes

Website `html.spec.whatwg.org`[?]

HTML

HTML

Inspector

Search HTML

`<!DOCTYPE html>`
`<html class="client-js vector-feature-language-in-header-enabled vector-fans-ready ve-`
`available vector-feature-toc-pinned-clientpref-0" lang="en" dir="ltr" data-lt-`
`installed="true"> <event> <scroll> <overflow>`
`</html>`

`<body class="skin--responsive skin-vector skin-vector-search-vue mediawiki_wiki HTML skin-`
`vector-2022 action-view uls-dialogs-sticky-hide"> <event>`
`<div id="mw-aria-live-region" class="mw-aria-live-region" aria-live="polite"></div>`
`<a class="mw-jump-link" href="#bodyContent">Jump to content</a> </div> </div>`
`<div class="vector-header-container"></div> </div> </div>`
`<div class="mw-page-container"></div>`
`</div class="rt-overlaid"></div>`

html:client-js:vector-feature-language-... > body.skin--responsive.skin-vector.skin-v...

Rules Layout Computed Changes Compatibility Fonts Animations

Filter Styles

element {}

@media screen {

html, body {} {

font-family: sans-serif;

}

@media screen {

html, body {} {

height: 100%;

}

@media screen {

body {} {

background-color: var(--background-color-neutral-subtle #f8f9fa);

color: var(--color-base #202122);

}

body {} {

margin: 0;

}

Inherited from html

@media screen {

body {} {

1

Errors, Warnings, Info, Logs, Debug

CSS XHR Requests

load.php:1:91107

Unknown property 'widows'. Declaration dropped.

load.php:1:91116

Unknown pseudo-class or pseudo-element '-moz-focus-inner'. Ruleset ignored due to bad selector.

load.php:1:155246

Unknown pseudo-class or pseudo-element '-moz-focus-inner'. Ruleset ignored due to bad selector.

HTML:7:848

Unknown pseudo-class or pseudo-element '-moz-focus-inner'. Ruleset ignored due to bad selector.

HTML:20:3839

Unknown pseudo-class or pseudo-element '-moz-focus-inner'. Ruleset ignored due to bad selector.

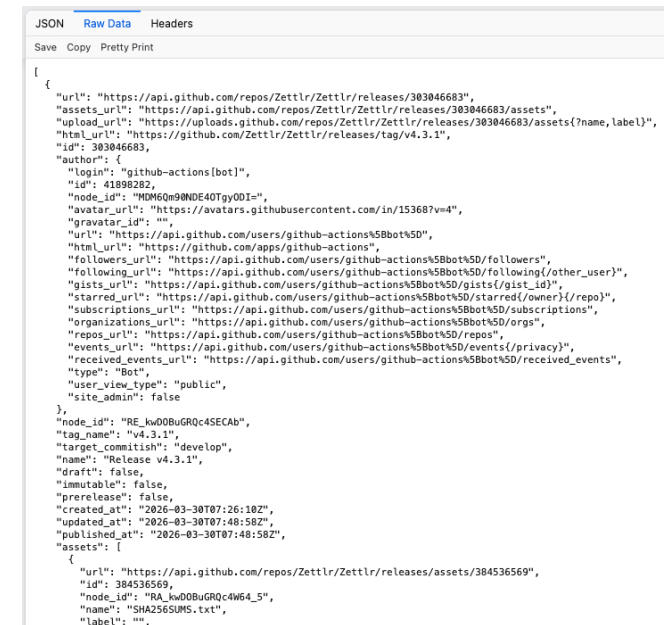
HTML:6:24786

Intro to APIs

How computers talk to each other

Application Programming Interface

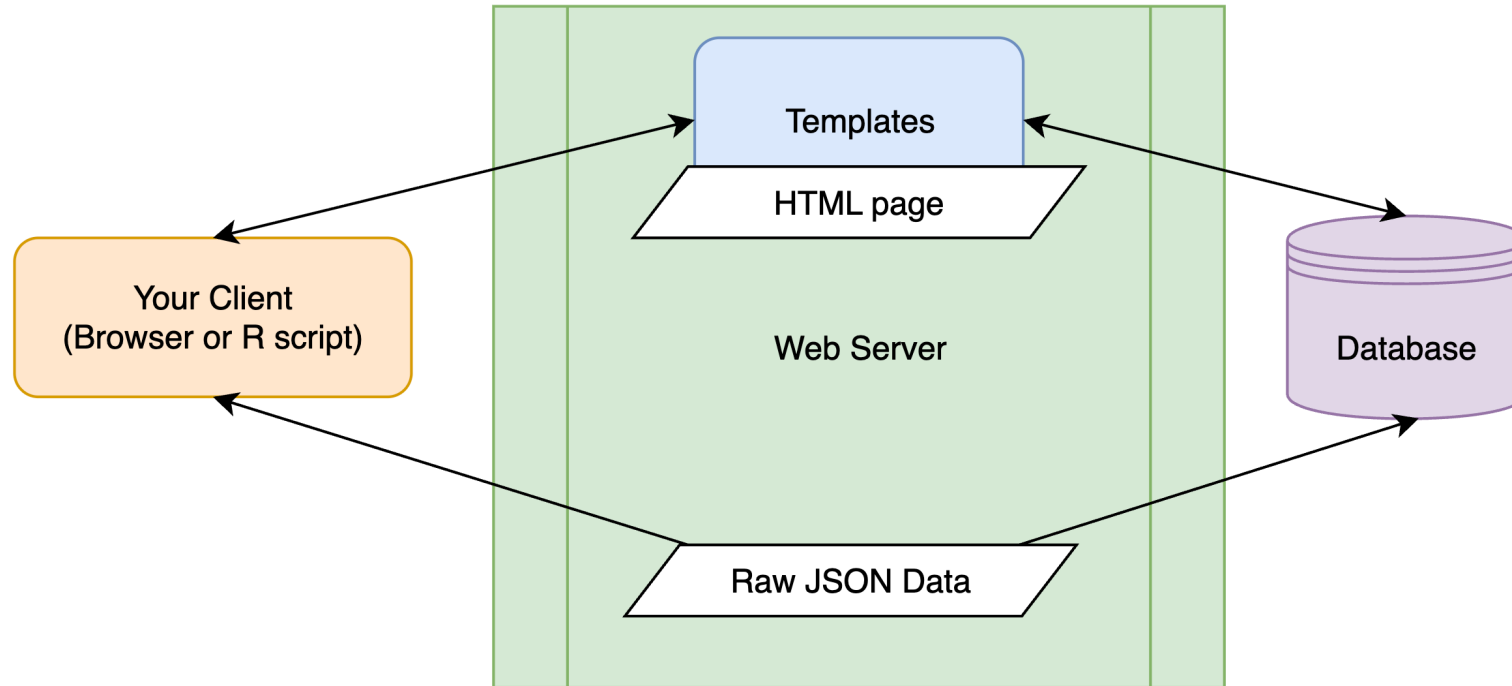
- An API is a set of endpoints that, when queried, will send back data in a machine-readable format
- Try it out: Visit the following URL in your browser and see what happens!
- <https://api.github.com/repos/Zettlr/Zettlr/releases>
- Compare that to the “regular” website:
- <https://github.com/Zettlr/Zettlr/releases>



```
JSON Raw Data Headers
Save Copy Pretty Print

[
  {
    "url": "https://api.github.com/repos/Zettlr/Zettlr/releases/383046683/assets",
    "assets_url": "https://api.github.com/repos/Zettlr/Zettlr/releases/383046683/assets",
    "upload_url": "https://uploads.github.com/repos/Zettlr/Zettlr/releases/383046683/assets{?name,label}",
    "html_url": "https://github.com/Zettlr/Zettlr/releases/tag/v4.3.1",
    "id": 383046683,
    "author": {
      "login": "github-actions[bot]",
      "id": 41898282,
      "node_id": "MDM6Om90NDk0TGY0ID=",
      "avatar_url": "https://avatars.githubusercontent.com/in/15368?v=4",
      "gravatar_id": "",
      "url": "https://api.github.com/users/github-actions%5Bbot%5D",
      "html_url": "https://github.com/apps/github-actions",
      "followers_url": "https://api.github.com/users/github-actions%5Bbot%5D/followers",
      "following_url": "https://api.github.com/users/github-actions%5Bbot%5D/following{/other_user}",
      "gists_url": "https://api.github.com/users/github-actions%5Bbot%5D/gists{/gist_id}",
      "starred_url": "https://api.github.com/users/github-actions%5Bbot%5D/starred{/owner}{/repo}",
      "subscriptions_url": "https://api.github.com/users/github-actions%5Bbot%5D/subscriptions",
      "organizations_url": "https://api.github.com/users/github-actions%5Bbot%5D/orgs",
      "repos_url": "https://api.github.com/users/github-actions%5Bbot%5D/repos",
      "events_url": "https://api.github.com/users/github-actions%5Bbot%5D/events{/privacy}",
      "received_events_url": "https://api.github.com/users/github-actions%5Bbot%5D/received_events",
      "type": "Bot",
      "user_view_type": "public",
      "site_admin": false
    },
    "node_id": "RE_kw00BuGRQc4SECAb",
    "tag_name": "v4.3.1",
    "target_commitish": "develop",
    "name": "Release v4.3.1",
    "draft": false,
    "immutable": false,
    "prerelease": false,
    "created_at": "2026-03-30T07:26:10Z",
    "updated_at": "2026-03-30T07:48:58Z",
    "published_at": "2026-03-30T07:48:58Z",
    "assets": [
      {
        "url": "https://api.github.com/repos/Zettlr/Zettlr/releases/assets/384536569",
        "id": 384536569,
        "node_id": "RA_kw00BuGRQc4W64_5",
        "name": "SHA256SUMS.txt",
        "label": ""
      }
    ]
  }
]
```

Visiting a Website vs an API



JavaScript Object Notation

- Most APIs work with JSON format
 - (There is also SOAP XML, which is an older enterprise standard)
- JSON is short for JavaScript Object Notation
- An agreed-upon way to encode data in a machine-readable form

```
1  [  
2    {  
3      "type": "car",  
4      "in_production": true,  
5      "features": ["doors", "windows", "wheels"]  
6    },  
7    {  
8      "type": "steam-engine",  
9      "in_production": false,  
10     "features": ["works with coal", "needs tracks"]  
11   },  
12   {  
13     "type": "plane",  
14     "in_production": true,  
15     "features": ["fast", "dirty", "convenient"]  
16   }  
17 ]
```

JavaScript Object Notation

- Types:
 - Arrays: `[1, 2, 3]`
 - Objects: `{ "key": "value" }`
 - Strings: `"steam-engine"`
 - Numbers: `420`
 - Booleans: `true`

The diagram illustrates the structure of a JSON array containing three objects. The code is as follows:

```
1  [  
2    {  
3      "type": "car",  
4      "in_production": true,  
5      "features": ["doors", "windows", "wheels"]  
6    },  
7    {  
8      "type": "steam-engine",  
9      "in_production": false,  
10     "features": ["works with coal", "needs tracks"]  
11   },  
12   {  
13     "type": "plane",  
14     "in_production": true,  
15     "features": ["fast", "dirty", "convenient"]  
16   }  
17 ]
```

Annotations with arrows:

- Object**: Points to the opening curly brace `{` on line 2.
- Property/key**: Points to the string `"type"` on line 3.
- Array (list)**: Points to the closing square bracket `]` on line 17.
- Value (another list)**: Points to the array `["fast", "dirty", "convenient"]` on line 15.

JSON in R

- The most common operations with JSON are to parse it and to encode it (turn it from R's data format into JSON or vice versa)
- One package: `jsonlite`
- Since you'll be mostly reading JSON, you can turn JSON into R data using `fromJSON`

```
install.packages("jsonlite")
library(jsonlite)

json_data <- '{"name": "John", "age": 30, "city": "New York"}'

# Parse JSON data
parsed_data <- fromJSON(json_data)

# Print the parsed data
print(parsed_data)
```

JSON in R

- The most common operations with JSON are to parse it and to encode it (turn it from R's data format into JSON or vice versa)
- One package: `jsonlite`
- Since you'll be mostly reading JSON, you can turn JSON into R data using `fromJSON`

Data	
▼ parsed_data	List of 3
\$ name: chr "John"	
\$ age : int 30	
\$ city: chr "New York"	

JSON in R

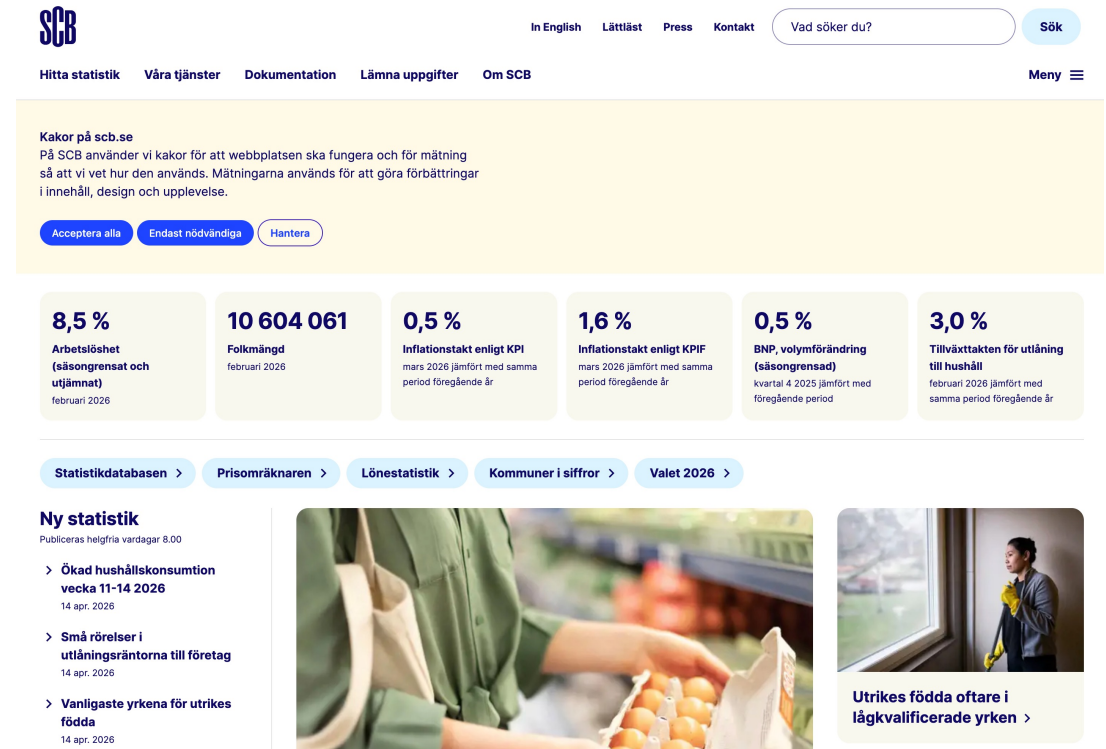
- Requesting API data works almost the same as requesting HTML data

```
# An endpoint that will give you JSON
url ← "https://api.github.com/repos/Zettlr/Zettlr/releases"
# Request just like with HTML
response ← GET(url)
# JSON is still just text
json_data ← content(response, as = "text")
# But this time we can immediately parse it
parsed_data ← fromJSON(json_data)
```

parsed_data		30 obs. of 23 variables	
\$ url	: chr	"https://api.github.com/repos/Zettlr/Zettlr/releases/303046683" "https://api.github.com/repos...	
\$ assets_url	: chr	"https://api.github.com/repos/Zettlr/Zettlr/releases/303046683/assets" "https://api.github.co...	
\$ upload_url	: chr	"https://uploads.github.com/repos/Zettlr/Zettlr/releases/303046683/assets/{name,label}" "http...	
\$ html_url	: chr	"https://github.com/Zettlr/Zettlr/releases/tag/v4.3.1" "https://github.com/Zettlr/Zettlr/rele...	
\$ id	: int	303046683 299711006 294231376 288972791 280446256 276404595 272878723 271537142 268257669 265...	
\$ author	: 'data.frame':	30 obs. of 19 variables:	
.. \$ login	: chr	"github-actions[bot]" "github-actions[bot]" "github-actions[bot]" "github-actions[bot]" "..."	
.. \$ id	: int	41898282 41898282 41898282 41898282 41898282 41898282 41898282 41898282 41898282 418982...	
.. \$ node_id	: chr	"MDM6Qm90NDE4OTgyODI=" "MDM6Qm90NDE4OTgyODI=" "MDM6Qm90NDE4OTgyODI=" "MDM6Qm90NDE4OTgyO...	
.. \$ avatar_url	: chr	"https://avatars.githubusercontent.com/in/15368?v=4" "https://avatars.githubusercontent.com/...	
.. \$ gravatar_id	: chr	" " " " " " " " " " " " " " "	
.. \$ url	: chr	"https://api.github.com/users/github-actions%5Bbot%5D" "https://api.github.com/users/gi...	
.. \$ html_url	: chr	"https://github.com/apps/github-actions" "https://github.com/apps/github-actions" "http...	
.. \$ followers_url	: chr	"https://api.github.com/users/github-actions%5Bbot%5D/followers" "https://api.github.com...	
.. \$ following_url	: chr	"https://api.github.com/users/github-actions%5Bbot%5D/following{/other_user}" "https://...	
.. \$ gists_url	: chr	"https://api.github.com/users/github-actions%5Bbot%5D/gists{/gist_id}" "https://api.git...	
.. \$ starred_url	: chr	"https://api.github.com/users/github-actions%5Bbot%5D/starred{/owner}/{repo}" "https://...	
.. \$ subscriptions_url	: chr	"https://api.github.com/users/github-actions%5Bbot%5D/subscriptions" "https://api.githu...	
.. \$ organizations_url	: chr	"https://api.github.com/users/github-actions%5Bbot%5D/orgs" "https://api.github.com/usee...	
.. \$ repos_url	: chr	"https://api.github.com/users/github-actions%5Bbot%5D/repos" "https://api.github.com/usu...	
.. \$ events_url	: chr	"https://api.github.com/users/github-actions%5Bbot%5D/events{/privacy}" "https://api.gi...	
.. \$ received_events_url	: chr	"https://api.github.com/users/github-actions%5Bbot%5D/received_events" "https://api.git...	
.. \$ type	: chr	"Bot" "Bot" "Bot" "Bot" ...	
.. \$ user_view_type	: chr	"public" "public" "public" "public" ...	
.. \$ site_admin	: logi	FALSE FALSE FALSE FALSE FALSE FALSE ...	
\$ node_id	: chr	"RE_kwDOBuGRQC4SEcAB" "RE_kwDOBuGRQC4R3Toe" "RE_kwDOBuGRQC4RiZ1q" "RE_kwDOBuGRQC4ROV_3" ...	
\$ tag_name	: chr	"v4.3.1" "v4.3.0" "v4.2.1" "v4.2.0" ...	
\$ target_commitish	: chr	"develop" "master" "master" "master" ...	

APIs in R

- Some APIs are popular
- For those APIs, there can be packages that automate a lot of the navigation for you
- Example: Statistics Sweden uses an API server called PXWeb
- Måns Magnusson (Uppsala) has written an R-package that automatically navigates the API for you



Authentication

Getting to the spicy data

Authentication

- More and more data is locked behind accounts and credentials
- We always need an account to access it
 - Facebook, Instagram, TikTok, ...
- Usually using a Username/Password combination
- Often with 2FA/TOTP

Please log in first.

email@example.com

Password

[Forgot password?](#)

Login

Authentication

- Once you have logged in, you are usually in a session
- That session is bound to the specific app → if you log in to a page using your browser, and then request the same website from an R script, that script is not logged in!

Authentication

- Authentication is usually handled via the headers of requests and responses

Example POST login:

```
POST /login HTTP/1.1  
Host: example.com
```

```
username=johndoe&password=super-secret
```

Authentication

- Authentication is usually handled via the headers of requests and responses

Example POST login response:

HTTP/1.1 200 OK

Set-Cookie: __Secure-session=PXrfKpaMtPMTFu79HHQHiULJx4Z4oZy28gbxQJazy0U%3D; Max-Age=604800; Domain=example.com; Path=/; Expires=Tue, 21 Apr 2026 12:19:20 GMT; HttpOnly; Secure; SameSite=Strict

Authentication

- When you perform a regular login via a website in R, you will have to save that cookie and send it with every following request to let the server know that you are still the same person

Example POST login response:

HTTP/1.1 200 OK

Set-Cookie: __Secure-session=PXrfKpaMtPMTFu79HHQHiULJx4Z4oZy28gbxQJazy0U%3D; Max-Age=604800; Domain=example.com; Path=/; Expires=Tue, 21 Apr 2026 12:19:20 GMT; HttpOnly; Secure; SameSite=Strict

Authentication

- When you query an API, doing the whole POST-Request, and Cookie-handling is a bit cumbersome
- That's why authentication to an API is usually handled using API keys.
- You simply provide such a key using what is known as an Authentication bearer header.

Example API request:

```
GET /api/data HTTP/1.1  
Host: example.com
```

```
Authorization: Bearer Ym9zY236Ym9zY28=
```

Storing Data

“Never throw away raw data”

Storing Data

- You should always save websites to disk
 - Only request a page once
 - Allows you to work offline
 - Saves resources
 - Allows for transparency (did you do a mistake somewhere, or was the website just wrong?)

```
# Fetch the Wikipedia page on Web Scraping
url <- "https://en.wikipedia.org/wiki/Web\_Scraping"
response <- GET(url)
html <- content(response, as = 'text')
# Write it to a file "wikipedia.html"
writeLines(html, "wikipedia.html")
# At a later point, you can read it in:
lines <- readLines('wikipedia.html')
# Turn the lines into a single string
html <- paste0(lines, collapse = '\n')
```

Conclusion

Conclusion

- Today we have talked about:
 - HTTP
 - Requests and Responses
 - Verbs
 - Status Codes
 - Body
 - Requesting Websites in R
 - Encoding
 - HTML
 - APIs
 - JSON
 - Authentication
- Tomorrow we will focus on how to extract data from HTML

Any Questions?

Thank you!